

Your code editor does this 1000 times a second

Parenthesis Check

"({[]})" -> Valid ✓

"({)})" -> Invalid ✗

{ () }

```
function test() {  
    if (arr[i] == "(") {  
        console.log("open");  
    }  
    { ← never closed  
}
```

matched matched unmatched = invalid

LeetCode 20: Valid Parentheses

Every close bracket must match the most recent unmatched open

Question

Given a string `s` containing just the characters `(` `)` `[` `]` `{` `}`, determine if the input string is valid.

A string is valid if:

1. Open brackets are closed by the **same type**.
2. Open brackets are closed in the **correct order**.
3. Every close bracket has a **matching open** of the same type.

{ } Input / Output

Input: `s = "()"`

Output: `true`

Input: `s = "()[]{}"`

Output: `true`

Input: `s = "(]"`

Output: `false (wrong type)`

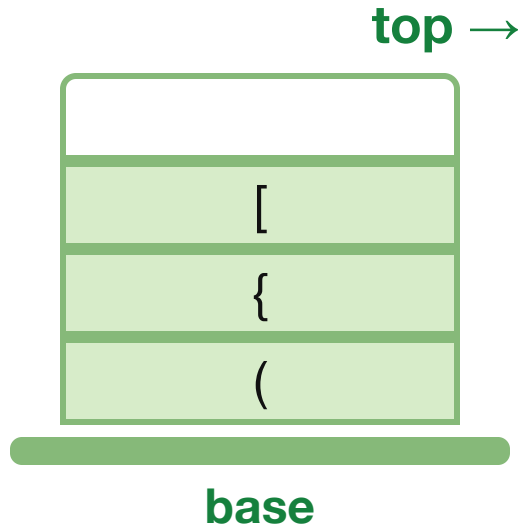
Input: `s = "([)]"`

Output: `false (wrong order)`

How Stack Solves It: Push Open, Pop on Close

Last open must match next close = LIFO

`s = "({[]})"`



Algorithm + Code

- if char is ({ [→ push
- else → if empty **or** top ≠ matching → **false**
- pop()
- return stack empty

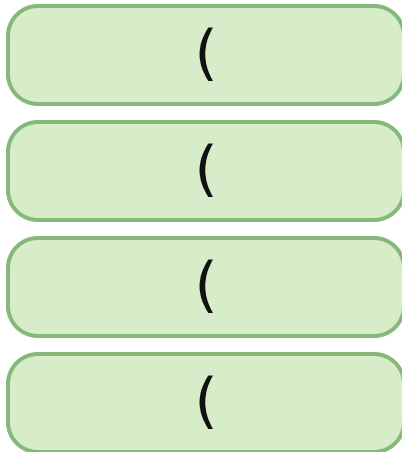
Example trace → characters processed one by one



Complexity + Takeaway

One pass, one stack

Stack Diagram



$O(n)$

Complexity Analysis

Time Complexity

— we visit each char once

$O(n)$

Space Complexity

— worst case all opens

$O(n)$

Push / Pop

$O(1)$

Takeaway: Last open must match next close → LIFO = Stack. Push opens, pop on close, check peek() and empty(). Pattern for all bracket-matching problems.